

Personal Reflection

Telco Churn Machine Learning Pipeline

Dana Bolden

Professor Rao's midterm feedback is what actually changed how I built this project. Going in, I assumed algorithm selection was the part that mattered most: pick the right classifier, tune it well, done. What this pipeline taught me instead is that a model's predictive power is bound by how clean the data is and how well the features are built before any algorithm ever sees them.

Tracking down a data corruption problem that nearly derailed the pipeline was the hardest part of this project. The first time I ran One-Hot Encoding, my feature matrix exploded past 3,000 columns. Something was clearly wrong. A closer audit turned up the culprit: hidden spaces inside the TotalRevenue column were making pandas read real financial numbers as categorical text. Coercing those strings back into numerics and imputing the resulting nulls with the column median was frustrating work, but it was the first time in this project I felt like I was actually debugging data, not just running someone else's checklist.

Building a strict 70/15/15 stratified split and applying SMOTE only to the training set was a harder concept to internalize than I expected. Saying "no leakage" is easy; enforcing it at every step of a pipeline is not. Watching the validation metrics hold steady once that was in place made the extra care worth it.

Feature engineering is where my thinking changed the most. My first instinct was to reach for simple unit conversions, like dividing tenure months by twelve, and Professor Rao's feedback made clear how little that actually told the model about a customer. Rebuilding Total_AddOn_Services and Total_Calls_All_Types as compound behavioral metrics, instead of single-column conversions, gave the models a real signal to separate loyal customers from churners, and it showed in the validation numbers.

The one thing I would do differently next time is spend more compute on hyperparameter tuning. My Soft Voting Ensemble ended up as the safer, more stable choice, with a 0.945 ROC-AUC on test data, but Gradient Boosting on its own showed real potential during validation that I never got to fully chase down: a proper RandomizedSearchCV pass on Gradient Boosting, not just the light grid search I ran on Random Forest, is probably where the next gain is hiding.

What this project actually changed was not my code, but how I think about building a pipeline: less about writing something that runs, more about building something that would hold up once it left my notebook.